FASTDRINK S5-GI

Manuel d'Ingénierie Système & Spécification Complète

Architecture Microservices, Registres Matériels, Protocoles IPC et Analyse des Défaillances

---

DOCUMENTATION TECHNIQUE INTÉGRALE

Plateforme SunFounder PiCar-S (ARM64 Linux) & Jumeau Numérique Godot 4

Projet : FastDrink S5-GI
Système : PiCar-S / Raspberry Pi
Noyau : Linux RT / Sysfs / POSIX IPC
Langage : C++17 / C++20

Version du document : 2.4.0
Date : Novembre 2026
Statut : Spécification Formelle
Validation : Banc de Test & Godot

Document technique confidentiel élaboré pour la réalisation du véhicule autonome FastDrink S5-GI.

Contents

1 Architecture Globale & Interfaces Noyau / Processus **5**
- 1.1 Introduction et Contexte Système . . . 5
- 1.2 Topologie des Processus et Découpage Fonctionnel . . . 5
- 1.3 Mécanismes de Communication Inter-Processus (IPC) . . . 6
  - 1.3.1 Mémoire Partagée POSIX Globale (`/dev/shm/picar_shm`) . . . 6
  - 1.3.2 Bus Mémoire Pub/Sub par Anneau Circulaire (`MemoryShareComponent`) . . . 6
  - 1.3.3 Sockets de Domaine UNIX (`AF_UNIX` / `SOCK_STREAM`) . . . 6
- 1.4 Diagramme d'Architecture Système et Interfaces Noyau . . . 7
- 1.5 Couche d'Abstraction Matérielle (HAL) . . . 7

2 Pilotes Matériels, Registres I2C & Communication Noyau **8**
- 2.1 Sous-Système Matériel et Cartographie des Bus . . . 8
- 2.2 Détection Dynamique de la Base GPIO du Noyau (`sysfs`) . . . 8
- 2.3 Contrôleur PWM PCA9685 (Adresse I2C `0x40`) . . . 10
  - 2.3.1 Plan Mémoire des Registres PCA9685 . . . 10
  - 2.3.2 Configuration des Bits de Contrôle (`MODE1` et `MODE2`) . . . 10
  - 2.3.3 Formules Mathématiques de Calcul des Registres . . . 11
- 2.4 Pont en H Double TB6612FNG (Propulsion Arrière) . . . 12
- 2.5 Module Suiveur de Ligne 5 Voies TCRT5000 (Adresses `0x11` / `0x0B`) . . . 12
  - 2.5.1 Protocole de Lecture et Décodage des Trames . . . 12
  - 2.5.2 Transformation en Noirceur Continue et Calibration . . . 13
  - 2.5.3 Tolérance aux Pannes I2C . . . 13
- 2.6 Télémètre Ultrasonique HC-SR04 (GPIOs Sysfs 12 & 16) . . . 13
  - 2.6.1 Equation Acoustique et Gestion des Délais . . . 13
  - 2.6.2 Protection Contre les Opérations Bloquantes . . . 13
- 2.7 Diagramme Synthétique de l'Interfaçage Matériel . . . 13

3 Spécification Détaillée des Bibliothèques Partagées **15**
- 3.1 Organisation et Statut des Bibliothèques . . . 15
- 3.2 Protocole Binaire et En-tête de Trames (`header/`) . . . 15
  - 3.2.1 Structure `PacketHeader` . . . 15
  - 3.2.2 Drapeaux d'En-tête (`shared::headers::flags`) . . . 15
  - 3.2.3 Algorithme de Sérialisation et Contrôle d'Intégrité . . . 16
- 3.3 Segment Mémoire Partagée POSIX (`ipc/shared_memory.hpp`) . . . 16
- 3.4 Ensemble de Commandes & Réponses (`commands/` & `responses/`) . . . 16
- 3.5 Algorithmes de Contrôle et Cinématique (`control/`) . . . 16
  - 3.5.1 Profileur de Vitesse et Limiteur d'Accélération (`MotionProfiler`) . . . 16
  - 3.5.2 Asservissement Suivi de Trajectoire (`LineFollowerController`) . . . 18
- 3.6 Pipeline de Traitement du Signal (`filters/`) . . . 19
  - 3.6.1 Étage 1 : Filtre Médian Glissant à 5 Échantillons (`MedianFilter`) . . . 19
  - 3.6.2 Étage 2 : Filtre de Kalman 1D Compensé par la Vitesse . . . 19

3.7 Services d'Infrastructure Partagés (`libs/shared-services/`) . . . 19
3.7.1 Mémoire Pub/Sub par Anneau Circulaire (`shared::services::memory`) . 19
3.7.2 Serveur et Client Sockets IPC UNIX (`shared::services::ipc`) . . . 19
3.7.3 Gestionnaire de Configuration (`shared::services::config`) . . . 19

4 Spécification Exhaustive des Services Logiciels 20
4.1 `ServiceManager` (Superviseur Système & Init PID 1) . . . 20
4.1.1 Rôle Architectural et Responsabilités . . . 20
4.1.2 Boucle de Supervision et Paramètres Temporels . . . 20
4.1.3 Codes d'Erreur et Conditions de Déclenchement . . . 21
4.2 `FSM` (Machine à États Finis & Contrôle de Trajectoire) . . . 21
4.2.1 Rôle Architectural et Composants Internes . . . 21
4.2.2 Fréquence d'Exécution et Cadencement Déterministe . . . 21
4.2.3 Paramètres de Configuration (`fsm.conf`) . . . 22
4.2.4 Codes d'Erreur et Drapeaux Composant (`shared::fsm::flags`) . . . 22
4.3 `Motors` (Actionnement Propulsion & Direction) . . . 22
4.3.1 Rôle Architectural et Composants Internes . . . 22
4.3.2 Fréquence d'Exécution et Cadencement . . . 22
4.3.3 Mécanismes de Chien de Garde et Failsafe . . . 23
4.4 `Sensors` (Acquisition Frontale & Traitement du Signal) . . . 23
4.4.1 Rôle Architectural et Composants Internes . . . 23
4.4.2 Fréquence d'Échantillonnage et Isolation Acoustique . . . 23
4.4.3 Gestion des Anomalies Capteurs . . . 23
4.5 `PeripheralsService` (Interface Matérielle & Signalisation) . . . 23
4.5.1 Rôle Architectural et Interfaces . . . 23
4.5.2 Algorithme d'Anti-Rebond Matériel/Logiciel . . . 24
4.6 `TelemetryService` (Concentrateur & Télémétrie Temps Réel) . . . 24
4.6.1 Rôle Architectural et Diffusion Réseau . . . 24
4.7 `SimBridge` (Passerelle Jumeau Numérique Godot 4) . . . 24
4.7.1 Rôle Architectural et Interchangeabilité . . . 24

5 Machine à États Finis (FSM) & Analyse des Défaillances 26
5.1 Formalisation Mathématique de l'Automate Décisionnel . . . 26
5.2 Catalogue Complet des États (`shared::fsm::MachineStates`) . . . 26
5.3 Diagramme de Transition d'États FSM . . . 26
5.4 Matrice Complète des Défaillances et Déclencheurs de Sécurité . . . 26

6 Contraintes Temporelles, Fréquences & Analyse Temps Réel 31
6.1 Tableau Récapitulatif des Fréquences et Périodicités Système . . . 31
6.2 Stratégie d'Isolation Temporelle des Périphériques Lents . . . 31
6.3 Chronogramme d'Exécution Temps Réel . . . 31
6.4 Justification Mathématique des Limites Physiques FastDrink . . . 31
6.4.1 Démonstration du Seuil Critique d'Accélération ($a_{\max} \leq 0{,}735$ m/s^2) . . . 31
6.4.2 Quantum Maximal de Vitesse par Pas Temporel ($\Delta t = 20$ ms) . . . 33
6.4.3 Vitesse Maximale Admissible en Courbe ($R_{\min} = 120$ mm) . . . 33
6.4.4 Profil de Freinage Ciblé et Arrêt de Précision (± 30 mm) . . . 33

List of Figures

1.1 Architecture globale de la pile microservices FastDrink et interfaçage avec le noyau Linux. 7

2.1 Schéma fonctionnel des connexions physiques et adressage des périphériques. . . . 14

5.1 Diagramme de transition d'états complet de la machine FSM. 28

6.1 Chronogramme temps réel illustrant l'isolation de fréquence et l'absence d'inversion de priorité. 32

List of Tables

2.1 Cartographie matérielle des périphériques PiCar-S sous Linux. 9
2.2 Registres principaux du contrôleur PWM I2C PCA9685. 11
2.3 Table de vérité du pont en H double TB6612FNG. 12

3.1 Organisation détaillée de la structure mémoire partagée `RobotTelemetry`. 17

4.1 Gestion des défaillances par le ServiceManager. 21

5.1 Définition complète des états de la machine FSM FastDrink. 27
5.2 Matrice d'analyse des modes de défaillance, seuils et mécanismes de repli. 28

6.1 Table maîtresse des fréquences d'exécution et contraintes temporelles. 32

Chapter 1

Architecture Globale & Interfaces Noyau / Processus

1.1 Introduction et Contexte Système

Le projet FastDrink S5-GI vise la conception d'un véhicule autonome capable de naviguer de manière autonome sur un tracé balisé (ruban noir de largeur 18 mm) tout en maintenant l'intégrité dynamique d'un chargement fragile (une bille métallique reposant au sein d'une coupelle concave peu profonde).

La contrainte physique fondamentale impose une accélération totale (linéaire et centripète) strictement bornée :

$$\|\mathbf{a}\| = \sqrt{a_{\text{tangentiel}}^2 + a_{\text{centripète}}^2} \leq a_{\max} = 0{,}735 \text{ m/s}^2 \tag{1.1}$$

Tout dépassement de ce seuil critique conduit immédiatement au déversement de la bille par franchissement de la bordure ($\Delta h = 1{,}50$ mm pour un rayon de cuvette $r = 20$ mm).

Pour garantir la résilience, le déterminisme temps réel et la testabilité unitaire, le logiciel est articulé autour d'une **architecture microservices modulaire en C++17**, s'exécutant directement sous Linux (noyau ARM64 sur Raspberry Pi ou x86_64 en simulation Godot). Chaque brique fonctionnelle est isolée dans un processus indépendant possédant son propre espace d'adressage virtuel, protégeant l'ensemble de la pile contre les fautes de segmentation locales.

1.2 Topologie des Processus et Découpage Fonctionnel

La pile logicielle se décompose en sept services autonomes, orchestrés par un superviseur système faisant office de gestionnaire de cycle de vie :

1. `ServiceManager` **(PID 1 / Init)** : Initialise les points de montage virtuels du noyau (`/dev`, `/proc`, `/sys`, `/dev/shm`), configure l'interface réseau Ethernet statique, lance le démon SSH Dropbear et supervise l'ensemble des démons applicatifs avec réarmement automatique.

2. `FSM` **(Finite State Machine / Cœur Décisionnel)** : Exécute la logique de haut niveau, le régulateur de suivi de trajectoire (PD barycentrique), le profileur cinématique trapézoïdal et les manœuvres d'évitement d'obstacle et de marche arrière guidée.

3. `Motors` **(Contrôle d'Actionnement)** : Pilote le contrôleur PWM/Servo PCA9685 via le bus I2C et les ponts en H TB6612 via les GPIOs Sysfs. Intègre un filtre de limitation de pente angulaire (slew-rate limiter) pour la direction.

4. `Sensors` **(Acquisition et Traitement du Signal)** : Échantillonne le module suiveur de ligne à 5 phototransistors TCRT5000 sur bus I2C et le télémètre ultrasonique HC-SR04 via GPIOs à déclenchement impulsionnel. Applique un pipeline de filtrage numérique à double étage (médian glissant + Kalman 1D / EMA).

5. `PeripheralsService` **(Interface Homme-Machine et Événements)** : Scrute les boutons poussoirs physiques (Start, WiFi) avec anti-rebond matériel/logiciel, module le buzzer piézo-électrique et diffuse les événements d'initialisation sur socket Unix.

6. `TelemetryService` **(Concentrateur de Diagnostic et Diffusion)** : Agrège l'état global du robot depuis la mémoire partagée, effectue la surveillance dynamique des violations d'accélération, publie un flux TCP unifié et diffuse des trames UDP broadcast sur le réseau local.

7. `SimBridge` **(Passerelle Jumeau Numérique Godot 4)** : Établit une passerelle réseau TCP bidirectionnelle (port 9000) avec le moteur physique Godot Engine, permettant l'interchangeabilité parfaite entre robot physique et simulateur sans modifier une seule ligne du code de décision.

1.3 Mécanismes de Communication Inter-Processus (IPC)

Pour assurer des transferts à latence sub-microseconde sans surcharge système, trois canaux de communication spécialisés coexistent :

1.3.1 Mémoire Partagée POSIX Globale (`/dev/shm/picar_shm`)

Ce segment de mémoire partagée POSIX (`shm\_open`, `mmap`) stocke la structure `RobotTelemetry`. Il permet un échange *zero-copy* bidirectionnel entre la perception (`Sensors`), la décision (`FSM`) et l'actionnement (`Motors`). La cohérence des données est protégée par un verrou d'exclusion mutuelle inter-processus POSIX (`pthread\_mutex\_t`) initialisé avec l'attribut `PTHREAD\_PROCESS\_SHARED`. Un compteur de battements de cœur (`heartbeat`) 64 bits est incrémenté périodiquement par la navigation pour servir de chien de garde matériel/logiciel.

1.3.2 Bus Mémoire Pub/Sub par Anneau Circulaire (`MemoryShareComponent`)

En complément de la SHM globale, une infrastructure Pub/Sub sans verrou basée sur des anneaux circulaires mémoire (`libs/shared-services/memory`) permet à chaque service d'émettre des trames typées horodatées (`SensorsTelemetry`, `MotorsComponentTelemetry`, `FSMTelemetry`, `ServoTelemetry`) à destination du concentrateur de télémétrie.

1.3.3 Sockets de Domaine UNIX (`AF_UNIX` / `SOCK_STREAM`)

Des sockets locaux créés dans `/tmp/` assurent le transport fiable des ordres asynchrones, des signaux d'activation et de reconfiguration dynamique en vol :

- `/tmp/fsm.sock` : Contrôle et télécommande du FSM (ex: `trigger\_start`, configuration de gains).
- `/tmp/motors.sock` : Décalage de trim (`STEERING\_OFFSET`), inversion de polarité.
- `/tmp/sensors.sock` : Paramètres de filtrage (`FILTER\_ALPHA`, seuils ADC).
- `/tmp/peripherals.sock` : Diffusion des événements boutons poussoirs et sonneries buzzer.

1.4 Diagramme d'Architecture Système et Interfaces Noyau

La Figure 1.1 illustre l'organisation modulaire, le découplage des espaces mémoire, les chemins de communication IPC et l'interaction avec le sous-système matériel via les interfaces du noyau Linux.

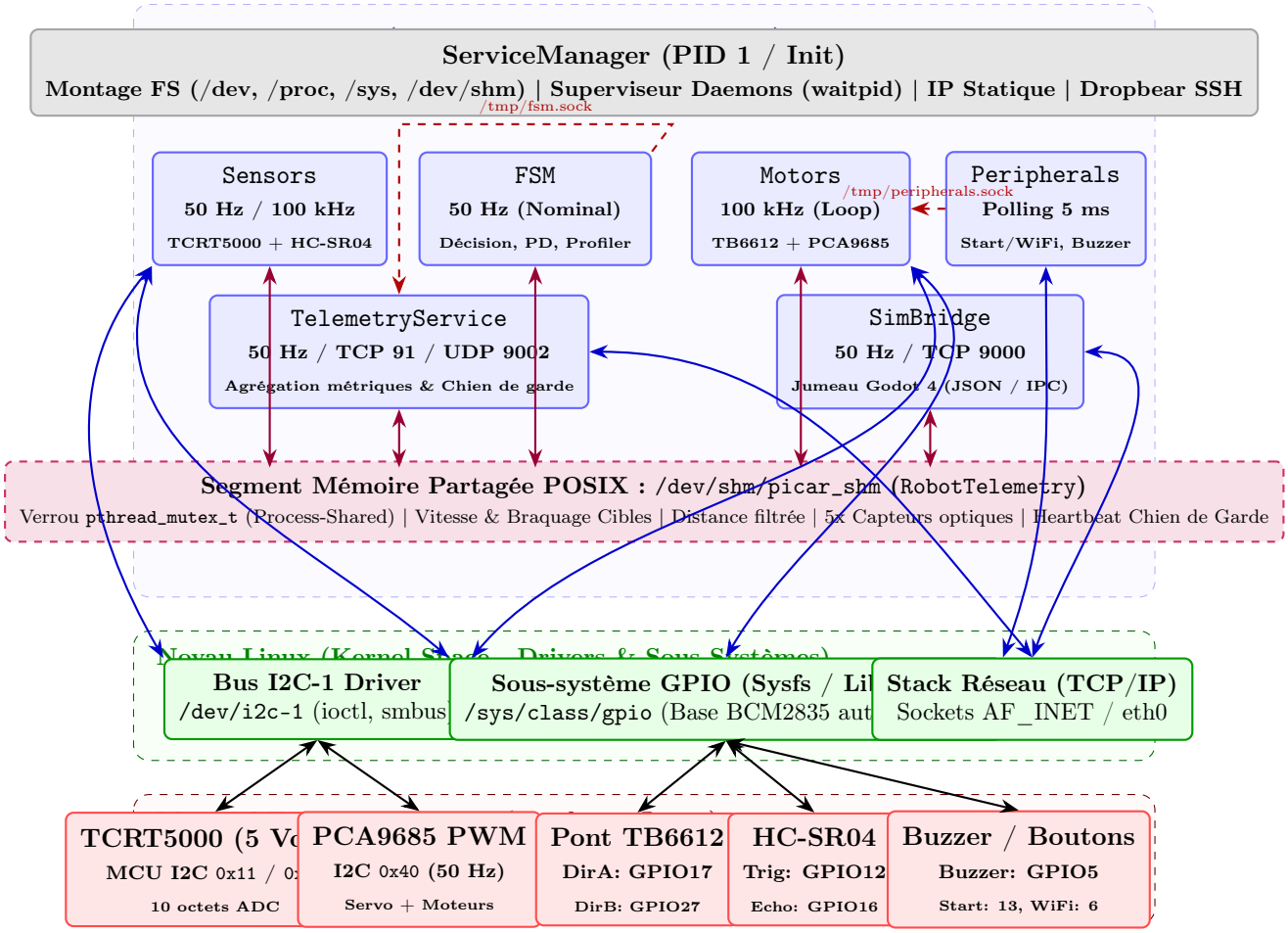


Figure 1.1: Architecture globale de la pile microservices FastDrink et interfaçage avec le noyau Linux.

1.5 Couche d'Abstraction Matérielle (HAL)

Conformément à la spécification technique **S10** (symétrie algorithmique), les couches décisionnelles du véhicule ne dépendent d'aucune bibliothèque matérielle directe (`sysfs`, `smbus`, `wiringPi`). Toutes les interactions matérielles sont encapsulées derrière deux interfaces virtuelles pures :

- `IMotorDriver` : Définit les fonctions d'actionnement `set\_speed(float)`, `set\_steering(float)`, `set\_steering\_offset(float)` et l'arrêt d'urgence `emergency\_stop()`.
- `ISensorDriver` : Définit les méthodes d'acquisition `read\_raw\_ultrasonic\_cm()`, `read\_line\_senso` et `read\_line\_darkness(float[5])`.

Cette conception garantit qu'en mode simulation, le service `SimBridge` peut injecter les grandeurs physiques virtuelles dans la SHM POSIX sans qu'aucun changement ne soit nécessaire dans l'exécutable `FSM`.

Chapter 2

Pilotes Matériels, Registres I2C & Communication Noyau

2.1 Sous-Système Matériel et Cartographie des Bus

Le robot SunFounder PiCar-S repose sur une carte fille de puissance (Robot HATS) connectée au connecteur d'extension GPIO 40 broches du Raspberry Pi. L'interfaçage matériel est pris en charge par le noyau Linux standard à travers deux mécanismes principaux du noyau :

1. **Le pilote de périphérique caractère I2C (`/dev/i2c-1`)** : Permet la communication sérielle synchrone avec le contrôleur PWM PCA9685, le microcontrôleur du suiveur de ligne et le convertisseur analogique-numérique PCF8591.

2. **Le sous-système de contrôle GPIO en espace utilisateur (`/sys/class/gpio`)** : Offre un contrôle bas niveau déterministe des signaux discrets pour la commande de direction des ponts en H, les signaux impulsionnels du télémètre ultrasonique et les entrées/sorties d'IHM (boutons et buzzer).

Le Tableau 2.1 récapitule l'assignation matérielle complète des périphériques, leurs adresses physiques et les broches BCM associées.

2.2 Détection Dynamique de la Base GPIO du Noyau (`sysfs`)

Dans les versions récentes du noyau Linux (notamment sur les kernels 5.x et 6.x pour Raspberry Pi 3, 4 et Zero 2W), la numérotation absolue des broches GPIO sous `/sys/class/gpio` n'est plus garantie de débuter à 0. Selon le pilote SoC activé (`bcm2835-gpio`, `pinctrl-bcm2711` ou `pinctrl-rp1`), la base peut être décalée à 512 ou toute autre valeur entière.

Pour garantir une portabilité absolue sans recompilation, la classe `PiCarHatsSensorDriver` et le service `PeripheralsService` implémentent un algorithme de détection automatique de la base GPIO :

```
static int detect_gpio_base() {
    DIR* dir = opendir("/sys/class/gpio");
    if (!dir) return 0;
    struct dirent* entry;
    int found_base = 0;
    while ((entry = readdir(dir)) != nullptr) {
        if (strncmp(entry->d_name, "gpiochip", 8) == 0) {
            char label_path[256];
            snprintf(label_path, sizeof(label_path), "/sys/class/gpio/%s/label",
    entry->d_name);
            int lfd = open(label_path, O_RDONLY);
            if (lfd >= 0) {
```

Périphérique	Interface	Adresse / Broche	Fréquence	Direction	Rôle Fonctionnel
PCA9685	I2C-1	`0x40` (`0x70`)	50 Hz PWM	OUT	Générateur PWM (Servo direction + Vitesse moteurs)
Suiveur Ligne	I2C-1	`0x11` (alt `0x0B`)	50 Hz / 100 kHz	IN	Lecture ADC 10 bits des 5 phototransistors TCRT5000
PCF8591 ADC	I2C-1	`0x48`	Sur demande	IN	Mesure analogique photorésistances & tension batterie
TB6612 DirA	Sysfs GPIO	BCM 17	Statique	OUT	Sens de rotation Moteur Gauche (1 = AV, 0 = AR)
TB6612 DirB	Sysfs GPIO	BCM 27	Statique	OUT	Sens de rotation Moteur Droit (1 = AV, 0 = AR)
HC-SR04 Trig	Sysfs GPIO	BCM 12	50 Hz (10 μs)	OUT	Déclenchement de la salve acoustique ultrasonique
HC-SR04 Echo	Sysfs GPIO	BCM 16	50 Hz	IN	Mesure de durée du signal d'écho réfléchi
Buzzer Piézo	Sysfs GPIO	BCM 5	200 Hz polling	OUT	Signalisation sonore (bips de démarrage, wifi, alerte)
Bouton WiFi	Sysfs GPIO	BCM 6	200 Hz ($\geq$ 10 ms)	IN	Bascule du point d'accès / client WiFi (Active LOW)
Bouton Start	Sysfs GPIO	BCM 13	200 Hz ($\geq$ 10 ms)	IN	Déclencheur physique du départ de course (Active LOW)

Table 2.1: Cartographie matérielle des périphériques PiCar-S sous Linux.

```
                char label[64] = {0};
                ssize_t n = read(lfd, label, sizeof(label) - 1);
                close(lfd);
                if (n > 0 && (strstr(label, "bcm2835") || strstr(label, "3
    f200000.gpio") || strstr(label, "fe200000.gpio"))) {
                    char base_path[256];
                    snprintf(base_path, sizeof(base_path), "/sys/class/gpio/%s/
    base", entry->d_name);
                    int bfd = open(base_path, O_RDONLY);
                    if (bfd >= 0) {
                        char base_str[32] = {0};
                        ssize_t bn = read(bfd, base_str, sizeof(base_str) - 1);
                        close(bfd);
                        if (bn > 0) {
                            found_base = std::atoi(base_str);
                            break;
                        }
                    }
                }
            }
        }
    }
    closedir(dir);
    return found_base;
}
```

Toute broche physique BCM_x est ainsi mappée sur le numéro de descripteur système réel $\mathrm{actual_pin} = \mathrm{found_base} + \mathrm{BCM}_x$.

2.3 Contrôleur PWM PCA9685 (Adresse I2C 0x40)

Le PCA9685 est un générateur PWM 16 canaux à résolution 12 bits (4096 pas) piloté par bus I2C. Il dispose d'un oscillateur interne typique à 25 MHz et d'un registre de prédivision d'horloge (PRESCALE).

2.3.1 Plan Mémoire des Registres PCA9685

Le Tableau 2.2 décrit les registres exploités par le pilote d'actionnement PiCarHatsMotorDriver (libs/shared-headers/hal/picar_i2c_registers.hpp).

2.3.2 Configuration des Bits de Contrôle (MODE1 et MODE2)

- **Bits de MODE1 :**
  - RESTART (0x80) : Autorise le redémarrage logique avec réactivation des sorties.
  - AUTO_INC (0x20) : Active l'auto-incrémentation automatique des adresses de registres lors des écritures séquentielles par bloc I2C.
  - SLEEP (0x10) : Place l'oscillateur interne en mode basse consommation. **Obligatoire** pour autoriser l'écriture dans le registre PRESCALE.
  - ALLCALL (0x01) : Répond aux trames adressées sur 0x70.
- **Bits de MODE2 :**
  - OUTDRV (0x04) : Configure les 16 sorties en structure **Totem-Pole** (Push-Pull, 5V/0V) plutôt qu'en drain ouvert, assurant des fronts montants francs pour le pilotage des servomoteurs RC et des entrées PWM du TB6612.

Registre	Adresse	Nom Symbolique	Description & Rôle
MODE1	0x00	picar::i2c::pca9685::MODE1	Registre de mode 1 (Mise en veille, auto-incrémentation, redémarrage).
MODE2	0x01	picar::i2c::pca9685::MODE2	Registre de mode 2 (Configuration des sorties : Totem-Pole vs Open-Drain).
SUBADR1-3	0x02 – 0x04	SUBADR1, SUBADR2, SUBADR3	Sous-adresses I2C secondaires (non utilisées).
ALLCALLADR	0x05	ALLCALLADR	Adresse d'appel général I2C (défaut 0x70).
LED0_ON_L	0x06	LED0_ON_L	Octet de poids faible du tick de passage à l'état HAUT du canal 0.
LED0_ON_H	0x07	LED0_ON_H	Octet de poids fort (bits 0–3) du tick ON du canal 0.
LED0_OFF_L	0x08	LED0_OFF_L	Octet de poids faible du tick de passage à l'état BAS du canal 0.
LED0_OFF_H	0x09	LED0_OFF_H	Octet de poids fort (bits 0–3) du tick OFF du canal 0.
ALL_LED_ON_L	0xFA	ALL_LED_ON_L	Commande globale ON de l'ensemble des 16 canaux.
ALL_LED_OFF_H	0xFD	ALL_LED_OFF_H	Commande globale OFF (Bit 4 = Forçage à 0V pour Failsafe).
PRESCALE	0xFE	PRESCALE	Registre de prédivision d'horloge fixant la fréquence PWM globale.

Table 2.2: Registres principaux du contrôleur PWM I2C PCA9685.

2.3.3 Formules Mathématiques de Calcul des Registres

1. Calcul du Prescaler d'Horloge

Pour obtenir une fréquence de commande de servomoteurs RC standard de $f_{\text{PWM}} = 50$ Hz à partir de l'horloge $f_{\text{osc}} = 25$ MHz et d'une résolution de 4096 points :

$$\text{prescale} = \text{round}\left(\frac{f_{\text{osc}}}{4096 \times f_{\text{PWM}}}\right) - 1 = \text{round}\left(\frac{25 \times 10^6}{4096 \times 50}\right) - 1 = 122 - 1 = 121 \quad (\texttt{0x79}) \tag{2.1}$$

2. Conversion d'Angle de Braquage en Ticks PWM

Le servomoteur de direction est alimenté sur une période $T = 20$ ms (soit 4096 ticks).

- Neutre ($0°$) $\rightarrow \tau = 1{,}50$ ms $\implies$ ticks $= \frac{1{,}50}{20} \times 4096 \approx 307$ ticks.
- Braquage Gauche maximal ($-45°$) $\rightarrow \tau = 1{,}00$ ms $\implies$ ticks ≈ 205 ticks.
- Braquage Droit maximal ($+45°$) $\rightarrow \tau = 2{,}00$ ms $\implies$ ticks ≈ 410 ticks.

L'équation d'interpolation affine bornée implémentée dans `steering\_angle\_to\_pwm` est :

$$\text{ticks}(\theta) = \text{clamp}\left(307{,}0 + \frac{\text{clamp}(\theta, -45°, +45°)}{45{,}0} \times 102{,}0,\ 205,\ 410\right) \tag{2.2}$$

3. Conversion du Pourcentage de Vitesse en Ticks PWM

La puissance moteur $[0{,}0\%, 100{,}0\%]$ est transformée en rapport cyclique de 0 à 4095 :

$$\text{ticks}(P) = \text{clamp}(|P|, 0{,}0, 100{,}0) \times 40{,}95 \tag{2.3}$$

4. Coupure d'Urgence Immédiate (Failsafe Matériel)

Pour désexciter instantanément tous les canaux PWM sans effectuer 16 transactions individuelles, le pilote injecte la valeur `0x10` dans le registre `ALL\_LED\_OFF\_H` (`0xFD`) :

```
void PiCarHatsMotorDriver::emergency_stop() {
    m_i2c.write_byte_data(picar::i2c::pca9685::ALL_LED_OFF_H, 0x10); // Bit 4 = Full OFF
}
```

2.4 Pont en H Double TB6612FNG (Propulsion Arrière)

Le circuit intégré Toshiba TB6612FNG commande les deux moteurs à courant continu du train arrière. La vitesse de rotation est modulée par les canaux PWM 4 (Moteur Gauche) et 5 (Moteur Droit) du PCA9685. Le sens de rotation est contrôlé par deux broches logiques directes du Raspberry Pi :

- **Moteur Gauche** (`DirA`) : BCM 17 (`/sys/class/gpio/gpio17/value`).
- **Moteur Droit** (`DirB`) : BCM 27 (`/sys/class/gpio/gpio27/value`).

Le Tableau 2.3 présente la table de vérité opérationnelle appliquée par le pilote.

Consigne Vitesse	DirA (BCM 17)	DirB (BCM 27)	PWM (CH 4 & 5)	Comportement V
$v > 0$	1 (HIGH)	1 (HIGH)	Ticks proportionnels	Marche Avant Nor
$v < 0$	0 (LOW)	0 (LOW)	Ticks proportionnels	Marche Arrière G
$v = 0$	Inchangé	Inchangé	0 tick	Roue libre / Arrêt
Arrêt Urgence	0 (LOW)	0 (LOW)	0 tick (Forcé `0xFD`)	Coupure Totale d'Ali

Table 2.3: Table de vérité du pont en H double TB6612FNG.

2.5 Module Suiveur de Ligne 5 Voies TCRT5000 (Adresses `0x11` / `0x0B`)

Le capteur de détection de trajectoire comporte cinq phototransistors infrarouges TCRT5000 alignés transversalement sur une barrette frontale, espacés d'environ 19 mm. Un microcontrôleur embarqué échantillonne les tensions analogiques de réflexion et met à disposition un registre I2C de lecture continue.

2.5.1 Protocole de Lecture et Décodage des Trames

La lecture est initiée par une lecture en bloc de 10 octets consécutifs à partir du registre `REG\_START` (`0x00`) :

$$\text{Trame I2C} = \begin{bmatrix} H_0 & L_0 & H_1 & L_1 & H_2 & L_2 & H_3 & L_3 & H_4 & L_4 \end{bmatrix} \tag{2.4}$$

Chaque capteur $i \in \{0, 1, 2, 3, 4\}$ dispose d'une mesure analogique sur 10 bits ($0 \leq \text{ADC}_i \leq 1023$) recomposée selon :

$$\text{ADC}_i = (H_i \ll 8) \mid L_i \tag{2.5}$$

2.5.2 Transformation en Noirceur Continue et Calibration

Sur un revêtement blanc réfléchissant, les photodiodes reçoivent un flux lumineux élevé, générant une tension et une valeur ADC élevées ($\text{ADC}_{\text{blanc}} \approx 80$ à 140). Sur le ruban isolant noir absorbant, le signal chute drastiquement ($\text{ADC}_{\text{noir}} \approx 15$ à 40).

La fonction `adc\_to\_darkness` projette de manière continue la lecture brute sur l'intervalle [0,0, 1,0] (où 1,0 désigne le noir absolu) :

$$D_i = \text{clamp}\left(\frac{\text{ADC}_{\text{blanc},i} - \text{ADC}_i}{\text{ADC}_{\text{blanc},i} - \text{ADC}_{\text{noir},i}},\ 0{,}0,\ 1{,}0\right) \tag{2.6}$$

Les seuils de calibration peuvent être personnalisés individuellement par capteur via le fichier persistant `/etc/picar\_line\_calibration` ou les variables d'environnement `PICAR\_LINE\_BLACK\_ADC` et `PICAR\_LINE\_WHITE\_ADC`.

2.5.3 Tolérance aux Pannes I2C

En cas de déconnexion transitoire ou de parasites sur le bus I2C, le pilote tolère jusqu'à 2 lectures erronées consécutives en conservant la dernière valeur acquise valide. Si le compteur dépasse 3 erreurs consécutives (≈ 60 ms), le pilote applique un repli sécuritaire (*failsafe*) en forçant toutes les valeurs à zéro ($D_i = 0$), déclenchant ainsi immédiatement la gestion de perte de ligne par le FSM.

2.6 Télémètre Ultrasonique HC-SR04 (GPIOs Sysfs 12 & 16)

Le capteur HC-SR04 mesure la distance par temps de vol d'une onde acoustique à 40 kHz.

1. **Impulsion Trigger (BCM 12)** : Le pilote génère une impulsion haute calibrée d'au moins 10 μs sur la broche `actual\_trig`.
2. **Mesure Echo (BCM 16)** : La broche `actual\_echo` passe à l'état HAUT pendant la durée de propagation aller-retour.

2.6.1 Equation Acoustique et Gestion des Délais

La vitesse du son dans l'air à 20°C étant $c \approx 343$ m/s $= 0{,}0343$ cm/μs, la distance en centimètres est donnée par :

$$d_{\text{cm}} = \frac{\Delta t_{\mu\text{s}} \times c}{2} = \frac{\Delta t_{\mu\text{s}}}{58{,}3} \approx \frac{\Delta t_{\mu\text{s}}}{58} \tag{2.7}$$

2.6.2 Protection Contre les Opérations Bloquantes

Pour préserver les contraintes temps réel strictes sans bloquer le thread d'acquisition :

- **Chien de garde de front montant** : Si le signal `Echo` ne monte pas dans les 5 ms suivant l'impulsion Trigger, la mesure est avortée avec le code $-1{,}0$ cm (obstacle absent ou capteur déconnecté).
- **Chien de garde de front descendant** : La boucle d'attente s'interrompt automatiquement après 25 ms, ce qui correspond à la portée utile maximale du robot (≈ 430 cm).

2.7 Diagramme Synthétique de l'Interfaçage Matériel

La Figure 2.1 expose le câblage direct entre le microprocesseur BCM2835 et les périphériques de puissance et de détection.

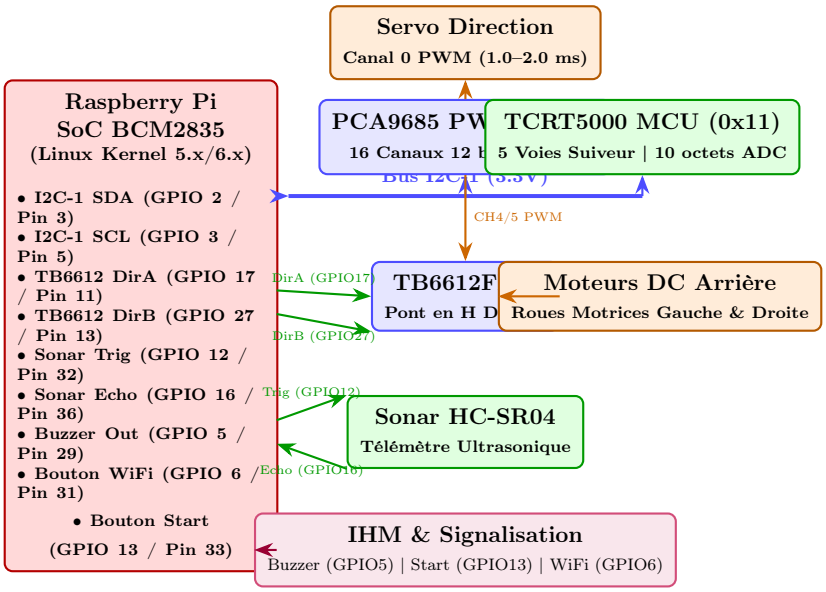


Figure 2.1: Schéma fonctionnel des connexions physiques et adressage des périphériques.

Chapter 3

Spécification Détaillée des Bibliothèques Partagées

3.1 Organisation et Statut des Bibliothèques

L'écosystème logiciel FastDrink repose sur deux ensembles de bibliothèques distincts :

1. `libs/shared-headers/` (**GELÉ / INTÉGRITÉ STRICTE**) : Contient l'ensemble des définitions de types fondamentaux, formats de trames binaires, drapeaux d'état, interfaces HAL et algorithmes de régulation mathématique. **Cette bibliothèque est strictement en lecture seule** afin de garantir la pérennité de l'interface IPC binaire entre tous les processus.
2. `libs/shared-services/` : Fournit l'implémentation opérationnelle des abstractions système partagées (serveur/abonnés de mémoire partagée cyclique, serveurs de sockets UNIX et gestionnaire de fichiers de configuration).

3.2 Protocole Binaire et En-tête de Trames (`header/`)

Toutes les communications sérialisées (sur socket UNIX ou TCP) partagent un en-tête binaire commun de 8 octets à alignement strict sur 32 bits, défini dans `header/header.hpp`.

3.2.1 Structure `PacketHeader`

```
struct PacketHeader {
    HeadersType type;         // 1 octet  : Command (0x01), Telemetry (0x02), Response (0x03)
    uint8_t     headerFlag;   // 1 octet  : Masque de bits (REPLY_REQUESTED, CRC_ENABLED)
    uint16_t    seqNumber;    // 2 octets : Numero de sequence sequentiel (detection de perte)
    uint32_t    payloadLength;// 4 octets: Longueur du payload (DOIT etre multiple de 4)
};
```

3.2.2 Drapeaux d'En-tête (`shared::headers::flags`)

- `REPLY\_REQUESTED` (`0x01`) : L'émetteur attend une confirmation immédiate (`Ack` ou `Nack`).
- `FIRE\_AND\_FORGET` (`0x02`) : Envoi asynchrone sans attente de réponse.
- `CRC\_ENABLED` (`0x04`) : La trame se termine par un mot de contrôle d'intégrité CRC-32 de 4 octets.

3.2.3 Algorithme de Sérialisation et Contrôle d'Intégrité

La fonction `serializePacket` applique un contrôle strict sur la longueur du payload :

$$\text{payloadLength} \pmod 4 = 0 \tag{3.1}$$

Si le drapeau `CRC\_ENABLED` est actif, la trame sérialisée est prolongée d'un mot de 32 bits calculé selon la norme IEEE 802.3 (`crc/crc_global.hpp`) avec le polynôme inverse standard :

$$P(x) = \texttt{0xEDB88320}, \quad \text{Valeur initiale} = \texttt{0xFFFFFFFF} \tag{3.2}$$

3.3 Segment Mémoire Partagée POSIX (`ipc/shared_memory.hpp`)

La structure `RobotTelemetry` constitue la mémoire centrale d'échange à latence sub-microseconde entre `Sensors`, `FSM` et `Motors`.

3.4 Ensemble de Commandes & Réponses (`commands/` & `responses/`)

Les commandes asynchrones transmises au format `PacketHeader` embarquent un sous-en-tête de commande `CommandsHeader` :

```
enum class ServiceID : uint8_t { TELEMETRY = 0x00, SENSORS = 0x01, MOTORS = 0x02
    , FSM = 0x03 };
enum class CommandID : uint8_t { Ping = 0x01, SetActive = 0x02, SetControlFlags
    = 0x03, SetConfigKey = 0x04 };

struct CommandsHeader {
    ServiceID serviceID;     // Service cible
    CommandID commandID;     // Type d'action
    uint16_t  payloadLen;    // Longueur des arguments
};
```

Pour la commande `SetConfigKey`, le payload est constitué de la structure `SetConfigPayload` :

```
struct SetConfigPayload {
    char key[64];            // Cle alphanumerique (ex: "LINE_KP", "
    MAX_ACCELERATION")
    float value;             // Valeur flottante a appliquer
};
```

En retour, le service interrogé émet une réponse `ResponseID::Ack` (`0x02`) ou `ResponseID::Nack` (`0x03`) accompagnée d'un code de rejet `NackResponse` :

```
struct NackResponse {
    uint16_t explanation_len;
    uint16_t nackCode;       // 1 = Erreur inconnue, 2 = Cle invalide, 3 =
    Valeur hors bornes
};
```

3.5 Algorithmes de Contrôle et Cinématique (`control/`)

3.5.1 Profileur de Vitesse et Limiteur d'Accélération (`MotionProfiler`)

Le module `MotionProfiler` implémente un générateur de rampe cinématique trapézoïdale garantissant le respect absolu de la contrainte FastDrink.

Champ Membre	Type	Taille	Description & Rôle Fonctionnel
`filtered\_dist`	`float`	4 octets	Distance acoustique filtrée (en cm, Médian + Kalman/EMA).
`speed`	`float`	4 octets	Vitesse estimée actuelle du véhicule (en m/s ou cm/s).
`line\_error`	`float`	4 octets	Écart barycentrique latéral de la ligne $[-1{,}0, +1{,}0]$.
`line\_sensors[5]`	`uint8_t[5]`	5 octets	État logique binaire binarisé des 5 TCRT5000 (1 = ligne noire).
`line\_darkness[5]`	`float[5]`	20 octets	Mesure de noirceur continue normalisée [0,0 = blanc, 1,0 = noir].
`target\_speed`	`float`	4 octets	Consigne de vitesse issue du MotionProfiler (écrite par FSM).
`target\_steering`	`float`	4 octets	Consigne de braquage en degrés (écrite par le PD FSM).
`steering\_offset`	`float`	4 octets	Trim logiciel de correction angulaire mécanique (en degrés).
`state`	`uint32_t`	4 octets	État machine FSM en cours (cast de `MachineStates`).
`emergency\_stop`	`bool`	1 octet	Drapeau d'arrêt d'urgence absolu (Failsafe logiciel immédiat).
`heartbeat`	`uint64_t`	8 octets	Compteur incrémenté à 50 Hz servant de chien de garde temporel.
`mtx`	`pthread_mutex_t`	40 octets	Mutex POSIX inter-processus (`PTHREAD\_PROCESS\_SHARED`).
`cfg\_max\_acceleration`	`float`	4 octets	Accélération maximale bornée ($a \leq 0{,}735$ m/s^2).
`cfg\_cruise\_speed`	`float`	4 octets	Vitesse de croisière nominale en ligne droite (m/s).
`cfg\_min\_curve\_speed`	`float`	4 octets	Vitesse plancher en courbe serrée $R = 120$ mm (m/s).
`cfg\_line\_kp`	`float`	4 octets	Gain proportionnel du régulateur de trajectoire.
`cfg\_line\_kd`	`float`	4 octets	Gain dérivé du régulateur de trajectoire.
`cfg\_curve\_safety\_factor`	`float`	4 octets	Coefficient de sécurité dynamique virage (0,85 à 0,93).
`cfg\_predictive\_brake\_gain`	`float`	4 octets	Gain feedforward sur la dérivée d'erreur pour anticipation.
`cfg\_version`	`uint32_t`	4 octets	Compteur de version incrémenté à chaque modification en vol.

Table 3.1: Organisation détaillée de la structure mémoire partagée `RobotTelemetry`.

1. Évolution Temporelle Discrète

À chaque pas temporel Δt, la vitesse v_k est ajustée selon la consigne désirée v_{cible} sans jamais dépasser le quantum $\Delta v_{\max} = a_{\max} \cdot \Delta t$:

$$v_k = \begin{cases} \min(v_{\text{cible}},\ v_{k-1} + a_{\max} \cdot \Delta t) & \text{si } v_{\text{cible}} > v_{k-1} \quad \text{(Phase d'accélération)} \\ \max(v_{\text{cible}},\ v_{k-1} - a_{\max} \cdot \Delta t) & \text{si } v_{\text{cible}} < v_{k-1} \quad \text{(Phase de décélération)} \end{cases} \tag{3.3}$$

2. Vitesse Limite en Virage (Modèle Cinématique d'Ackermann)

Soit $L = 0{,}14$ m l'empattement du véhicule et δ l'angle de braquage des roues directrices avant. Le rayon de courbure instantané R du centre de l'essieu arrière est :

$$R = \frac{L}{\tan(|\delta|)} \tag{3.4}$$

L'accélération centrifuge s'écrivant $a_c = \frac{v^2}{R} \leq a_{\max}$, la vitesse critique maximale admissible en virage est :

$$v_{\text{virage}}(\delta) = \eta_{\text{safety}} \cdot \sqrt{a_{\max} \cdot \frac{L}{\tan(|\delta|)}} \tag{3.5}$$

où $\eta_{\text{safety}} \in [0{,}85, 0{,}95]$ représente le facteur de sécurité paramétrable (`CURVE\_SAFETY\_FACTOR`).

3. Freinage Prédictif par Anticipation

Pour éviter les à-coups violents à l'entrée soudaine d'un virage, un terme de freinage prédictif feedforward atténue la consigne de vitesse proportionnellement à la vitesse de variation de l'erreur latérale :

$$v_{\text{adapt}} = v_{\text{virage}} - K_{\text{brake}} \cdot \left| \frac{de}{dt} \right| \tag{3.6}$$

3.5.2 Asservissement Suivi de Trajectoire (`LineFollowerController`)

L'orientation du train avant est régulée par un contrôleur proportionnel-dérivé-intégral (PID) barycentrique optimisé.

1. Calcul d'Écart Barycentrique Analogique

Chaque capteur $i \in \{0, 1, 2, 3, 4\}$ est affecté d'un coefficient spatial $W = \{-2{,}0, -1{,}0, 0{,}0, 1{,}0, 2{,}0\}$. L'erreur latérale normalisée $e \in [-1{,}0, +1{,}0]$ est calculée par pondération de la noirceur continue D_i :

$$e = \frac{\sum_{i=0}^{4} W_i \cdot D_i}{2{,}0 \cdot \sum_{i=0}^{4} D_i} \tag{3.7}$$

Si la somme totale $\sum D_i < 0{,}25$, le robot est considéré en perte totale de ruban.

2. Équation de Contrôle de l'Angle de Braquage

L'angle de braquage commandé au servomoteur (en degrés) est :

$$\theta(t) = K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt} + K_i \int e(\tau)\, d\tau \tag{3.8}$$

Le terme intégral n'est actif qu'au voisinage immédiat du centre ($|e| < 0{,}6$) afin de compenser les dérives mécaniques de parallélisme sans provoquer d'effet de saturation (*wind-up*) en virage prononcé.

3.6 Pipeline de Traitement du Signal (`filters/`)

Les lectures du capteur ultrasonique HC-SR04 subissent un traitement numérique à deux étages en cascade dans la classe `UltrasonicPipeline` :

3.6.1 Étage 1 : Filtre Médian Glissant à 5 Échantillons (`MedianFilter`)

Le filtre médian élimine strictement les bruits impulsifs, réflexions acoustiques multiples et lectures fantômes à 0 cm :

$$y_1[k] = \text{médiane}\Big(x[k], x[k-1], x[k-2], x[k-3], x[k-4]\Big) \tag{3.9}$$

3.6.2 Étage 2 : Filtre de Kalman 1D Compensé par la Vitesse

Le signal débruité alimente un filtre de Kalman 1D dont l'équation de prédiction intègre la vitesse réelle du véhicule :

$$\hat{d}_{k|k-1} = \hat{d}_{k-1|k-1} - v \cdot \Delta t \tag{3.10}$$

$$P_{k|k-1} = P_{k-1|k-1} + Q \tag{3.11}$$

$$K_k = \frac{P_{k|k-1}}{P_{k|k-1} + R} \tag{3.12}$$

$$\hat{d}_{k|k} = \hat{d}_{k|k-1} + K_k \cdot \Big(y_1[k] - \hat{d}_{k|k-1}\Big) \tag{3.13}$$

$$P_{k|k} = (1 - K_k) \cdot P_{k|k-1} \tag{3.14}$$

En mode allégé, le Kalman est remplacé par une moyenne mobile exponentielle (`LowPassEMA`) :

$$y[k] = \alpha \cdot y_1[k] + (1 - \alpha) \cdot y[k-1], \quad \text{avec } \alpha = 0{,}25 \tag{3.15}$$

3.7 Services d'Infrastructure Partagés (`libs/shared-services/`)

3.7.1 Mémoire Pub/Sub par Anneau Circulaire (`shared::services::memory`)

Les classes `MemoryShareComponentServer` et `MemoryShareComponentSubscriber` fournissent une communication multi-abonnés sans verrou basée sur un compteur de publication atomique 32 bits et un tampon circulaire en mémoire partagée. L'abonné vérifie si un nouvel échantillon est disponible en comparant son dernier index lu avec l'index de publication exposé par le serveur.

3.7.2 Serveur et Client Sockets IPC UNIX (`shared::services::ipc`)

La classe `ServiceIPCSocketServer` crée un socket d'écoute `AF\_UNIX` en mode **non-bloquant** (`O\_NONBLOCK`). La méthode `process\_pending\_commands()` utilise l'appel système `poll()` avec un délai d'attente nul (0 ms) pour consommer les requêtes en instance sans suspendre la boucle de contrôle temps réel critique.

3.7.3 Gestionnaire de Configuration (`shared::services::config`)

La classe `ServiceConfig` résout automatiquement le chemin du fichier de configuration en examinant séquentiellement : `/root/<conf>`, `./<conf>`, `../<conf>`, `../../<conf>` et `/etc/<conf>`. Elle offre l'écriture synchrone persistante sur disque pour sauvegarder les paramètres calibrés en cours de session.

Chapter 4

Spécification Exhaustive des Services Logiciels

4.1 `ServiceManager` (Superviseur Système & Init PID 1)

4.1.1 Rôle Architectural et Responsabilités

Le `ServiceManager` constitue le processus racine (`PID 1`) du système embarqué lorsque le Raspberry Pi démarre sur une image Linux minimale de production, ou un orchestrateur utilisateur en phase de développement. Ses missions sont :

1. Montage initial des pseudo-systèmes de fichiers virtuels indispensables au noyau : `devtmpfs` sur `/dev`, `proc` sur `/proc`, `sysfs` sur `/sys`, `tmpfs` sur `/dev/shm` et `/run`, `devpts` sur `/dev/pts`.
2. Réarmement et nettoyage préventif des ressources IPC résiduelles (`unlink("/dev/shm/picar_shm")`, `unlink("/tmp/*.sock")`).
3. Configuration réseau de l'interface de bouclage `lo` (127.0.0.1) et attribution d'une adresse IP statique sur le lien Ethernet filaire `eth0` (192.168.0.100).
4. Lancement et surveillance active du serveur SSH léger `dropbear` (port 22) pour la maintenance à distance.
5. Gestion du cycle de vie des cinq microservices opérationnels (`FSM`, `Sensors`, `Motors`, `TelemetryService`, `PeripheralsService`) avec réapparition automatique en cas de crash.

4.1.2 Boucle de Supervision et Paramètres Temporels

La boucle de surveillance s'exécute à une fréquence de **10 Hz** (période de scrutation de **100 ms**) via l'appel système `usleep(100000)` :

- **Collecte asynchrone des processus défunts** : Appel non bloquant `waitpid(-1, \&status, WNOHANG)`.
- **Délai de refroidissement de redémarrage (Cooldown)** : Un délai minimum de **100 ms** est imposé entre deux tentatives de relance d'un service défaillant.
- **Protection anti-emballement (Exponential Backoff)** : Si un service subit plus de 5 échecs consécutifs, le gestionnaire temporise la relance pendant au moins **2 000 ms** (2 secondes) pour éviter la saturation du CPU par des boucles de crash infinies.

- **Extinction propre (Graceful Shutdown)** : Lors de la réception de `SIGINT` ou `SIGTERM`, le superviseur envoie d'abord `SIGINT` à tous ses démons enfants, accorde une période de grâce de **1 500 ms**, puis élimine les processus récalcitrants par `SIGKILL` avant de nettoyer la mémoire partagée et les sockets.

4.1.3 Codes d'Erreur et Conditions de Déclenchement

Condition Détectée	Constat Système	Action & Code Associé
Arrêt inopiné d'un service	`WIFEXITED(status) == true`	Log `[SUPERVISOR]`, incrémente `restart\_count`, re-planifie l'exécution.
Mort par signal noyau	`WIFSIGNALED(status) == true`	Log du numéro de signal (`SIGSEGV`, `SIGABRT`), réarmement différé.
Instabilité répétée	restart_count ≥ 5 en < 5 s	Blocage temporaire 2 s, activation du drapeau d'anomalie système.
Câble Ethernet débranché	`check\_carrier(iface) == false`	Déréférencement de la route IP, log `[NET] link DOWN`.
Gel d'un processus enfant	Aucun battement depuis > 500 ms	Déclenchement du drapeau `PROCESS\_STALLED` (`0x0400`).

Table 4.1: Gestion des défaillances par le ServiceManager.

4.2 `FSM` (Machine à États Finis & Contrôle de Trajectoire)

4.2.1 Rôle Architectural et Composants Internes

Le service `FSM` héberge l'intelligence décisionnelle du robot. Il intègre quatre sous-composants majeurs :

1. `FsmNavigator` : Orchestre le régulateur de suivi de ligne optique (`LineFollowerController`), le profileur cinématique trapézoïdal (`MotionProfiler`) et la machine d'état de contournement (`ObstacleAvoidance`).
2. `FsmShmBridge` : Assure la lecture non bloquante des capteurs et l'écriture synchrone des consignes moteur/servo dans `/dev/shm/picar_shm`.
3. `FsmEventPipe` : Détecte l'impulsion de départ en provenance du bouton physique via `/tmp/peripherals.sock`.
4. `FsmIpcHandler` : Réceptionne les ordres de réglage dynamique sur `/tmp/fsm.sock`.

4.2.2 Fréquence d'Exécution et Cadencement Déterministe

La boucle principale s'exécute à **50 Hz** (période nominale $T = 20$ ms). Le respect de la cadence est assuré par un calcul d'échéance absolue anti-dérive (*anti-drift deadline pacing*) :

```
const auto period = std::chrono::nanoseconds(1000000000LL / m_hz);
auto next_deadline = std::chrono::steady_clock::now();
while (m_running.load()) {
    next_deadline += period;
    // ... execution de la logique temps reel ...
    std::this_thread::sleep_until(next_deadline);
}
```

4.2.3 Paramètres de Configuration (`fsm.conf`)

- `HZ` (50,0) : Fréquence d'itération de la boucle FSM.
- `MAX\_ACCELERATION` (0,735 m/s^2) : Accélération maximale admissible (Seuil FastDrink).
- `STRAIGHT\_CRUISE\_SPEED` (0,35 à 0,45 m/s) : Vitesse cible en ligne droite.
- `MIN\_CURVE\_SPEED` (0,14 à 0,17 m/s) : Vitesse plancher en courbe de rayon $R = 120$ mm.
- `LINE\_KP` (1,1453) et `LINE\_KD` (0,1416) : Coefficients proportionnel et dérivé du suivi de ligne.
- `OBSTACLE\_STOP_DIST\_CM` (20,0 cm) : Distance d'arrêt d'urgence devant un obstacle frontal.
- `REVERSE\_SPEED` (0,12 m/s) : Vitesse de recul guidé en cas de perte de ruban.

4.2.4 Codes d'Erreur et Drapeaux Composant (`shared::fsm::flags`)

- `DISCOVERABLE` (`0x0001`) : Service présent et connecté à la SHM.
- `ANOMALIES\_DETECTED` (`0x0002`) : Violation de contrainte ou comportement anormal détecté.
- `WARNING\_DETECTED` (`0x0004`) : Avertissement mineur (ex: perte momentanée de ligne).
- `CONFIG\_ERROR` (`0x0008`) : Fichier de configuration absent ou paramètre hors bornes.
- `DRIVER\_ERROR` (`0x0010`) : Défaillance d'accès à la mémoire partagée.
- `SOCKET\_ERROR` (`0x0020`) : Échec de création ou d'écoute sur `/tmp/fsm.sock`.
- `SHM\_ERROR` (`0x0040`) : Segment `/dev/shm/picar_shm` introuvable ou verrou bloqué.
- `STALLED` (`0x0080`) : Chien de garde capteurs expiré (> 250 ms).

4.3 `Motors` (Actionnement Propulsion & Direction)

4.3.1 Rôle Architectural et Composants Internes

Le service `Motors` transforme les consignes cinématiques $(v_{\text{target}}, \delta_{\text{target}})$ en signaux matériels physiques (PWM et états logiques GPIO). Il s'appuie sur :

1. `MotorActuator` : Instancie le pilote matériel `PiCarHatsMotorDriver` et applique un filtre de limitation de variation angulaire (*slew-rate limiter*) continu pour éviter les à-coups sur le servo de direction (175°/s en ligne droite, 300°/s en courbe).
2. `MotorShmBridge` : Lit les consignes de navigation dans la SHM et met à jour la télémétrie locale.
3. `MotorIpcHandler` : Réceptionne les ajustements de trim de direction sur `/tmp/motors.sock`.

4.3.2 Fréquence d'Exécution et Cadencement

La boucle d'actionnement tourne à très haute fréquence (**100 kHz** par défaut dans `motors.conf`), garantissant une latence minimale de réaction et une intégration temporelle continue du slew-rate limiter. La sortie PWM matérielle vers le servo et les moteurs est quant à elle cadencée à **50 Hz** par le contrôleur PCA9685.

4.3.3 Mécanismes de Chien de Garde et Failsafe

- **Chien de garde FSM (250 ms)** : Si le compteur `heartbeat` de la SHM ne progresse plus pendant 250 ms, le service force immédiatement la consigne de vitesse à 0,0 m/s (coupure automatique de sécurité).
- **Arrêt d'urgence immédiat** : Dès que `emergency\_stop` passe à `true`, le service invoque `emergency\_stop()` qui force les sorties du PCA9685 à 0V via le registre `ALL\_LED\_OFF\_H` (`0xFD`) et bascule les broches de direction TB6612 à l'état BAS.

4.4 `Sensors` (Acquisition Frontale & Traitement du Signal)

4.4.1 Rôle Architectural et Composants Internes

Le service `Sensors` effectue la capture de l'environnement physique et son pré-traitement numérique :

1. `SensorAcquisition` : Gère le pilote matériel `PiCarHatsSensorDriver` et exécute le pipeline de filtrage ultrasonique (`UltrasonicPipeline`).
2. `LineEstimator` : Calcule l'écart barycentrique continu $e \in [-1{,}0, +1{,}0]$ à partir des noirceurs analogiques des 5 TCRT5000.
3. `SensorShmBridge` : Écrit les grandeurs filtrées dans la mémoire partagée POSIX et publie la structure `SensorsTelemetry` sur l'anneau Pub/Sub.

4.4.2 Fréquence d'Échantillonnage et Isolation Acoustique

Alors que la boucle logicielle scrute la mémoire et le suiveur de ligne jusqu'à **100 kHz**, **l'acquisition ultrasonique est strictement isolée à 50 Hz** :

```
const uint64_t us_interval = (hz > 50) ? static_cast<uint64_t>(hz / 50) : 1;
if (loop_tick % us_interval == 0) {
    float raw = m_driver->read_raw_ultrasonic_cm();
    // Traitement du signal ...
}
```

Cette isolation évite de suspendre le thread de perception pendant les 5 à 25 ms nécessaires à la propagation acoustique de l'onde ultrasonore du HC-SR04.

4.4.3 Gestion des Anomalies Capteurs

- **Perte du bus I2C** : Si plus de 3 lectures I2C consécutives échouent (≈ 60 ms), le capteur de ligne passe en état de repli (toutes les noirceurs à 0) et positionne le drapeau `LINE\_LOST`.
- **Timeout acoustique** : Si aucun écho n'est reçu dans les 5 ms, la valeur $-1{,}0$ cm est retournée et `consecutive\_us\_errors` est incrémenté.

4.5 `PeripheralsService` (Interface Matérielle & Signalisation)

4.5.1 Rôle Architectural et Interfaces

Le service `PeripheralsService` contrôle les interactions physiques locales :

- **Bouton Start (GPIO 13)** : Déclenche la transition de l'état `Armed` vers le départ de la course.
- **Bouton WiFi (GPIO 6)** : Déclenche l'activation / bascule de l'interface réseau sans fil.

- **Buzzer (GPIO 5)** : Fournit des retours auditifs distincts (mélodie d'initialisation, bip de confirmation WiFi, tonalité de départ).
- **Socket IPC UNIX (`/tmp/peripherals.sock`)** : Serveur non-bloquant diffusant les notifications d'événements aux services abonnés (notamment `FSM` et `TelemetryService`).

4.5.2 Algorithme d'Anti-Rebond Matériel/Logiciel

La scrutation est cadencée à **200 Hz** (délai de poll de 5 ms). Un état de bouton n'est validé comme front actif que s'il demeure stable à l'état bas (*Active LOW*) pendant une fenêtre minimale de **10 ms** :

```
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(now -
    tracker.press_start).count();
if (duration >= DEBOUNCE_MIN_MS && !tracker.triggered) {
    tracker.triggered = true;
    on_trigger(); // Emission de l'evenement
}
```

4.6 `TelemetryService` (Concentrateur & Télémétrie Temps Réel)

4.6.1 Rôle Architectural et Diffusion Réseau

Le `TelemetryService` assure la surveillance centralisée de l'ensemble du robot :

1. **Agrégation SHM & Pub/Sub** : Échantillonne simultanément la SHM globale et les anneaux Pub/Sub de `Sensors`, `Motors`, `FSM` et `Servo`.
2. **Calcul Dynamique des Accélérations** : Dérive en temps réel la vitesse estimée ($a = \frac{\Delta v}{\Delta t}$) et comptabilise le nombre de violations du seuil critique FastDrink ($|a| > 0{,}785$ m/s^2).
3. **Odométrie** : Intègre la distance parcourue $d = \int |v|\, dt$.
4. **Chien de Garde Global** : Détecte le gel du compteur de heartbeat (> 500 ms) et lève le drapeau `PROCESS\_STALLED`.
5. **Serveur TCP (Port 91)** : Permet la connexion d'un banc de diagnostic graphique distant.
6. **Émetteur UDP Broadcast (Port 9002)** : Diffuse les trames de diagnostic en clair à 50 Hz sur le réseau local.
7. **Enregistreur USB (`UsbLogger`)** : Exporte les métriques de vol sur clé USB externe.

4.7 `SimBridge` (Passerelle Jumeau Numérique Godot 4)

4.7.1 Rôle Architectural et Interchangeabilité

Le service `SimBridge` assure l'interopérabilité complète avec le jumeau numérique sous Godot Engine 4 :

- **Mode SIMULATION** : Écoute sur le port TCP **9000**. Le client Godot (`network_client.gd`) transmet les données des capteurs virtuels (distances des `RayCast3D` et `ShapeCast3D`) que le bridge injecte directement dans `/dev/shm/picar_shm`. En retour, le bridge transmet à Godot les consignes de vitesse et d'angle de braquage calculées par `FSM`.

- **Mode VEHICLE (Réel)** : Le bridge lit la télémétrie des capteurs physiques et la transmet à Godot uniquement pour la visualisation 3D en temps réel, sans écraser les lectures capteurs matérielles.

La synchronisation est verrouillée à **50 Hz** (20 ms par cycle), correspondant rigoureusement au pas physique (`\_physics\_process`) du moteur de jeu.

Chapter 5

Machine à États Finis (FSM) & Analyse des Défaillances

5.1 Formalisation Mathématique de l'Automate Décisionnel

L'automate à états finis (`FSM`) régit le comportement autonome global du robot selon un sextuplet formel :

$$\mathcal{M} = \langle \mathcal{S},\ \Sigma,\ \mathcal{O},\ \delta,\ \lambda,\ s_0 \rangle \tag{5.1}$$

où :

- $\mathcal{S}$ représente l'ensemble des 15 états discrets (`MachineStates`).
- Σ est l'ensemble des événements d'entrée (télémétrie capteurs, distances acoustiques, noirceurs optiques, chiens de garde, commandes utilisateur).
- $\mathcal{O}$ est le vecteur de consigne d'actionnement en sortie : $\mathbf{u} = \begin{bmatrix} v_{\text{cible}} & \theta_{\text{cible}} & \text{emergency_flag} \end{bmatrix}^T$.
- $\delta : \mathcal{S} \times \Sigma \to \mathcal{S}$ est la fonction de transition d'état.
- $\lambda : \mathcal{S} \times \Sigma \to \mathcal{O}$ est la fonction de sortie cinématique.
- $s_0 =$ `Init` est l'état initial au démarrage.

5.2 Catalogue Complet des États (`shared::fsm::MachineStates`)

Le Tableau 5.1 décrit les 15 états définis dans `libs/shared-headers/fsm/machine\_states.hpp`.

5.3 Diagramme de Transition d'États FSM

La Figure 5.1 synthétise le graphe complet des transitions d'états, les conditions de garde temporelles et événementielles associées.

5.4 Matrice Complète des Défaillances et Déclencheurs de Sécurité

Le Tableau 5.2 répertorie l'ensemble des modes de défaillance prévus par le système, les conditions exactes de leur déclenchement ainsi que la procédure d'arrêt d'urgence exécutée.

État	Code Binaire	Nom Symbolique	Signification Opérationnelle & Actionneurs
S_0	0x00	Init	Initialisation des ressources matérielles, IPC et fichiers de configuration ($t < 0{,}1$ s). $v = 0, \theta = 0$.
S_1	0x01	Unarmed	Calibrage statique des capteurs optiques et repos ($t < 0{,}4$ s). $v = 0, \theta = 0$.
S_2	0x02	Armed	Système armé prêt pour le départ. Attente de l'impulsion Start ou du délai auto-start.
S_3	0x03	LineFollowingForward	Régulation nominale de trajectoire par PD barycentrique et profil de vitesse adaptatif.
S_4	0x04	CollisionRisk	Décélération progressive d'urgence bornée ($a \leq 0{,}735$ m/s^2) devant un obstacle ($d \leq 20$ cm).
S_5	0x05	Stopped	Véhicule complètement immobilisé ($v < 0{,}03$ m/s). Analyse de l'obstacle ou fin de course.
S_6	0x06	CollisionOccured	Impact matériel ou obstacle infranchissable. Coupure totale des moteurs.
S_7	0x07	RightDeviation	Manœuvre d'évitement latérale par la droite (+25°) à 0,15 m/s (contournement S4).
S_8	0x08	LeftDeviation	Manœuvre d'évitement latérale par la gauche (-25°) à 0,15 m/s.
S_9	0x09	Reverse	Marche arrière guidée autonome ($v = -0{,}12$ m/s, recul maximal < 150 mm - Spécification S5).
S_{10}	0x0A	LineSearching	Contre-braquage directionnel (-24°) pour recouper le ruban noir après évitement.
S_{11}	0x0B	LineDetected	Ruban noir ré-acquis ($D \geq 0{,}55$). Stabilisation de la trajectoire pendant 0,2 s.
S_{12}	0x0C	LineSearchingTimedOut	Échec de réacquisition de ligne après 4,0 s. Coupure d'urgence sécuritaire.
S_{13}	0x0D	Abort	Abandon de mission sur demande utilisateur ou opérateur distant. $v = 0, \theta = 0$.
S_{14}	0x0E	Failure	Arrêt d'urgence de sécurité (violation accélération, perte de capteurs, rupture watchdog).

Table 5.1: Définition complète des états de la machine FSM FastDrink.

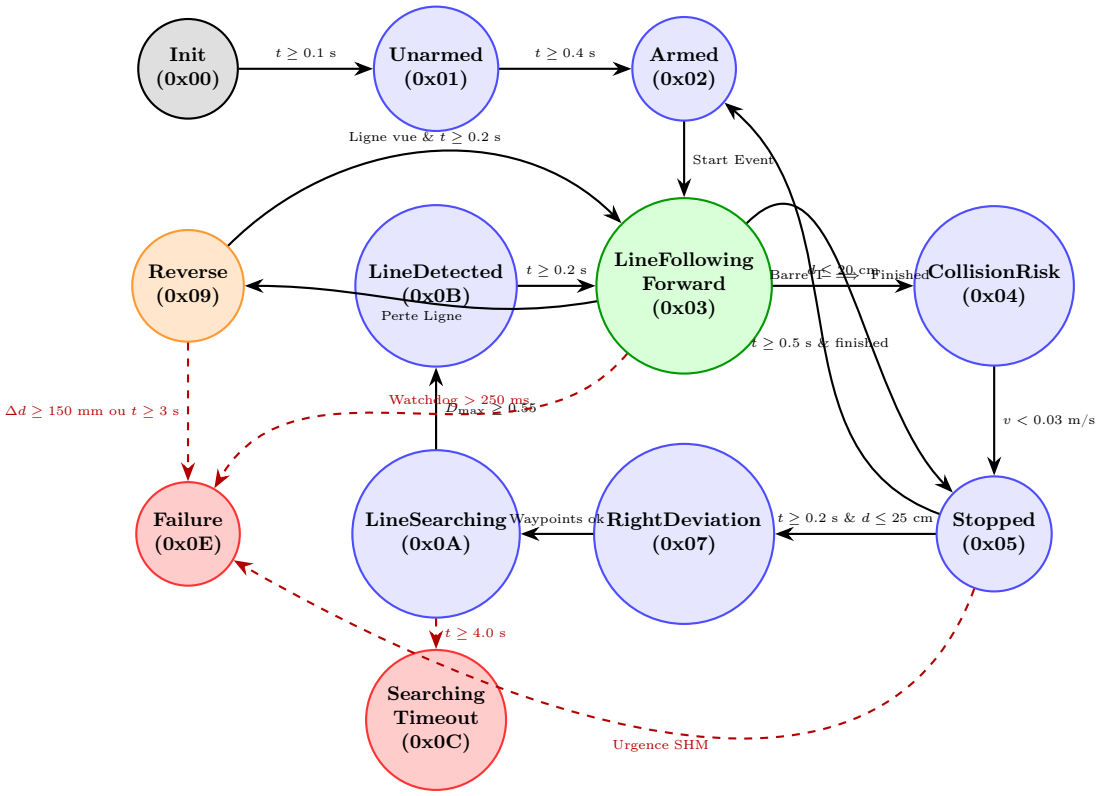


Figure 5.1: Diagramme de transition d'états complet de la machine FSM.

Table 5.2: Matrice d'analyse des modes de défaillance, seuils et mécanismes de repli.

Mode de Panne	**Condition de Déclenchement**	**Action Immédiate (Failsafe)**	**Comportement Système & Reprise**
Perte de Heartbeat Capteurs	Aucun incrément du compteur `heartbeat` de la SHM pendant ≥ 250 ms	FSM bascule immédiatement en `Failure` (S_{14}). $v = 0, \theta = 0$.	Coupure moteur matérielle via `ALL\_LED\_OFF\_H`. Reprise uniquement après réinitialisation complète.
Perte de Heartbeat Navigation	Le service `Motors` constate $> (hz \cdot 0{,}25)$ cycles sans variation de consigne	Moteurs coupent la vitesse ($v_{\text{target}} = 0{,}0$).	Protection contre l'emballement en cas de gel du FSM. Reprise dès rétablissement du flux.

Suite sur la page suivante...

Table 5.2 – *Suite de la page précédente*

Mode de Panne	**Condition de Déclenchement**	**Action Immédiate (Failsafe)**	**Comportement Système & Reprise**
Perte du Bus I2C Ligne	Échec de lecture continue I2C pendant ≥ 3 cycles (≈ 60 ms)	Forçage des noirceurs à zéro ($D_i = 0$), bascule en `Reverse` (S_9).	Le robot tente un recul autonome < 150 mm. Si toujours aucune ligne après 3 s, transition vers `Failure`.
Perte de Réflexion Ligne	Détection de ruban nulle $\sum_{i=0}^{4} D_i < 0{,}25$ en suivi nominal	Transition immédiate vers `Reverse` (S_9).	Vitesse inversée $v = -0{,}12$ m/s. Réalignement sur le ruban ou arrêt sécurité après 150 mm.
Échec Évitement d'Obstacle	Temps de recherche dans `LineSearching` (S_{10}) supérieur à 4,0 s	Transition vers `LineSearchingTimedOut` (S_{12}).	Levée de `emergency\_stop`, coupure des ponts en H. Intervention manuelle requise.
Gel Processus Système	Absence de mise à jour SHM détectée par TelemetryService > 500 ms	Positionnement du bit `PROCESS\_STALLED` (`0x0400`).	Notification sur console de télémétrie et diffusion du code d'anomalie sur le réseau.
Violation Accélération	Accélération estimée $\lvert a \rvert > 0{,}785$ m/s^2 ($a > a_{\max} + \epsilon$)	Incrémentation du compteur `accel\_violations`, alerte visuelle.	Alerte temps réel. Si rupture physique, signalement immédiat sur le flux télémétrique.

Suite sur la page suivante...

Table 5.2 – *Suite de la page précédente*

Mode de Panne	**Condition de Déclenchement**	**Action Immédiate (Failsafe)**	**Comportement Système & Reprise**
Crash Processus Fils	Signal noyau `SIGSEGV`, `SIGBUS` ou `SIGABRT` sur un service	Le superviseur `ServiceManager` collecte le code via `waitpid`.	Pause de refroidissement de 100 ms, puis respawn automatique du binaire (backoff 2 s si $\geq$ 5 crashs).
Timeout Télémètre Sonar	Absence de front montant sur GPIO Echo après 5 ms	Retourne la distance invalide $-1{,}0$ cm.	Rejet de l'échantillon par le filtre médian. Si persistant, signalement de défaillance capteur.

Chapter 6

Contraintes Temporelles, Fréquences & Analyse Temps Réel

6.1 Tableau Récapitulatif des Fréquences et Périodicités Système

La conception en microservices asynchrones impose un cadencement rigoureux de chaque boucle d'exécution afin de concilier la réactivité face aux événements imprévus et la bande passante limitée des bus matériels.

Le Tableau 6.1 synthétise les fréquences nominales, les périodes associées, les mécanismes de synchronisation et les chemins critiques de chaque composant de la pile FastDrink.

6.2 Stratégie d'Isolation Temporelle des Périphériques Lents

L'un des défis majeurs dans la programmation temps réel sous Linux consiste à éviter qu'une opération d'entrée-sortie lente ne bloque une boucle de contrôle rapide.

Le capteur ultrasonique HC-SR04 en est l'illustration parfaite : la mesure par écho sonore requiert entre 1 ms (pour un obstacle à 17 cm) et 25 ms (pour la portée maximale). Si cette lecture était exécutée directement à l'intérieur de la boucle rapide de `Sensors`, le thread subirait un gel imprévisible, rendant impossible la détection haute fréquence de la ligne optique.

La solution mise en œuvre repose sur une **isolation de fréquence synchrone par diviseur d'horloge** :

$$N = \left\lfloor \frac{f_{\text{loop}}}{f_{\text{ultrason}}} \right\rfloor = \left\lfloor \frac{100\,000}{50} \right\rfloor = 2\,000 \text{ cycles} \tag{6.1}$$

La mesure acoustique n'est déclenchée que lorsque loop_tick (mod N) = 0. Durant les 1999 autres cycles, le service réutilise la dernière distance filtrée tout en continuant d'échantillonner le ruban noir à pleine vitesse sans aucun retard.

6.3 Chronogramme d'Exécution Temps Réel

La Figure 6.1 illustre le cadencement relatif des processus sur un intervalle type de 40 ms (deux trames de contrôle à 50 Hz).

6.4 Justification Mathématique des Limites Physiques FastDrink

6.4.1 Démonstration du Seuil Critique d'Accélération ($a_{\max} \leq 0{,}735$ $\mathbf{m/s^2}$)

Considérons une bille de masse m reposant au fond d'une coupelle sphérique de profondeur utile $\Delta h = 1{,}50$ mm $= 0{,}0015$ m et de rayon horizontal bord-à-centre $d = 20{,}0$ mm $= 0{,}020$ m.

Composant / Service	Fréquence	Période (T)	Gigue Max.	Mécanisme	Chemin Critique / Rôle
`ServiceManager`	10 Hz	100 ms	±5 ms	`usleep()`	Supervision démons, lien Ethernet filaire, Dropbear SSH.
`FSM` (Contrôle)	**50 Hz**	**20 ms**	±0,5 ms	`sleep\_until`	Décision, régulateur PD, profileur de vitesse trapézoïdal.
`Motors` (Loop)	100 kHz	10 μs	±2 μs	`sleep\_until`	Polling SHM, slew-rate limiter de direction continu.
PCA9685 PWM	50 Hz	20 ms	Matériel	Hardware	Période PWM servomoteur RC (impulsions 1 à 2 ms).
`Sensors` (Loop)	100 kHz	10 μs	±2 μs	`sleep\_until`	Scrutation I2C ligne, calcul barycentrique, mise à jour SHM.
HC-SR04 Sonar	**50 Hz**	**20 ms**	±1 ms	Diviseur N	Impulsion 10 μs, écoute écho (5 à 25 ms max).
`PeripheralsService`	200 Hz	5 ms	±0,5 ms	`poll(5ms)`	Échantillonnage boutons (anti-rebond $\geq$ 10 ms), buzzer.
`TelemetryService`	50 Hz	20 ms	±1 ms	`sleep\_until`	Collecte SHM, dérivation d'accélération, UDP broadcast.
Console Télémétrie	5 Hz	200 ms	±10 ms	Modulo loop	Affichage synthétique des métriques sur terminal console.
`SimBridge` **(Godot)**	50 Hz	20 ms	±0,5 ms	Socket TCP	Synchronisation avec `\_physics\_process` de Godot 4.

Table 6.1: Table maîtresse des fréquences d'exécution et contraintes temporelles.

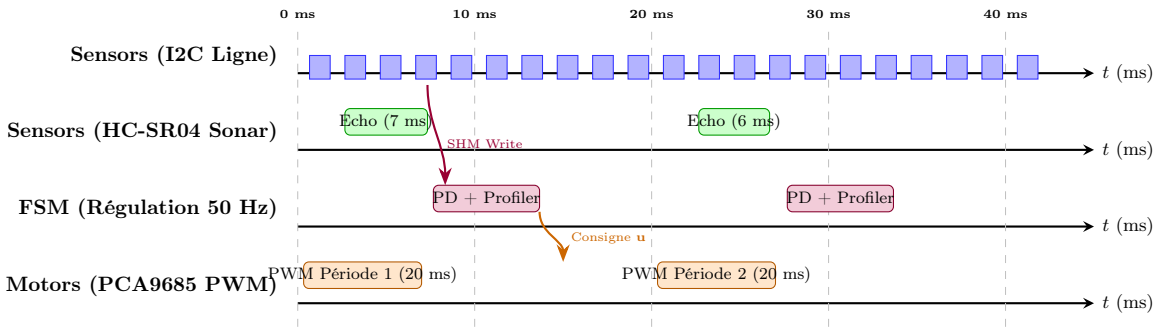


Figure 6.1: Chronogramme temps réel illustrant l'isolation de fréquence et l'absence d'inversion de priorité.

Le travail mécanique fourni par la force d'inertie lors d'une accélération linéaire a sur la distance d pour franchir la bordure s'écrit :

$$W_{\text{inertie}} = F_{\text{inertie}} \cdot d = (m \cdot a) \cdot d \tag{6.2}$$

L'énergie potentielle de pesanteur nécessaire pour faire monter la bille sur la hauteur Δh est :

$$\Delta E_p = m \cdot g \cdot \Delta h \tag{6.3}$$

À la limite du basculement, par le théorème de l'énergie mécanique ($W_{\text{inertie}} = \Delta E_p$) :

$$m \cdot a \cdot d = m \cdot g \cdot \Delta h \implies a = g \cdot \frac{\Delta h}{d} \tag{6.4}$$

En retenant l'accélération gravitationnelle standard $g = 9{,}80665$ m/s^2 :

$$a_{\max} = 9{,}80665 \times \frac{0{,}0015}{0{,}020} = 0{,}735498 \text{ m/s}^2 \approx \mathbf{0{,}735} \text{ m/s}^{\mathbf{2}} \tag{6.5}$$

Toute accélération instantanée supérieure à cette valeur éjecte irrémédiablement la bille.

6.4.2 Quantum Maximal de Vitesse par Pas Temporel ($\Delta t = 20$ ms)

Pour la boucle décisionnelle cadencée à 50 Hz ($\Delta t = 0{,}020$ s), le saut maximal de vitesse permis par le `MotionProfiler` entre deux itérations consécutives est rigoureusement borné par :

$$\Delta v_{\text{cycle}} \leq a_{\max} \cdot \Delta t = 0{,}735 \times 0{,}020 = 0{,}0147 \text{ m/s} \quad (1{,}47 \text{ cm/s par cycle}) \tag{6.6}$$

6.4.3 Vitesse Maximale Admissible en Courbe ($R_{\min} = 120$ mm)

Lors du franchissement d'un virage de rayon minimal imposé $R_{\min} = 120$ mm $= 0{,}12$ m (Spécification S3), l'accélération centripète $a_c = \frac{v^2}{R}$ ne doit pas excéder $a_{\max}$:

$$v_{\text{virage}} \leq \sqrt{a_{\max} \cdot R_{\min}} = \sqrt{0{,}735 \cdot 0{,}12} = \sqrt{0{,}0882} \approx \mathbf{0{,}297} \text{ m/s} \quad (29{,}7 \text{ cm/s}) \tag{6.7}$$

6.4.4 Profil de Freinage Ciblé et Arrêt de Précision (± 30 mm)

Pour satisfaire la spécification **S1** (immobilisation devant un obstacle à la distance cible $d_{\text{cible}} = 200$ mm avec tolérance stricte ± 30 mm), la vitesse résiduelle en fonction de la distance mesurée d est régie par la loi de décélération quadratique optimale :

$$v_{\text{freinage}}(d) = \sqrt{2 \cdot a_{\max} \cdot (d - d_{\text{cible}})} \tag{6.8}$$

Lorsque $|d - d_{\text{cible}}| \leq 2$ cm, la commande commute sur l'état `Stopped` sans choc mécanique, garantissant la parfaite stabilité de la bille.